3D Orbital Mechanics & Numerical Integration

Task 3: Centripetal Force & Euler Integration

Section A: Groups 3 & 8

St. John Baptist De La Salle Catholic School

March 20, 2026

Newton's Law of Universal Gravitation

The force governing our simulation is the mutual attraction between the Earth and the satellite.

Gravitational Force Formula:

$$F_g = \frac{GM_E m_s}{r^2} \tag{1}$$

In our VPython code, we treat Earth as a fixed point and calculate the force vector acting on the satellite at every time step (dt).

Centripetal Acceleration (a_c)

For an object in a curved path, acceleration points toward the center of curvature.

Definition: The "center-seeking" acceleration that changes the velocity's direction.

$$a_c = \frac{v^2}{r} \tag{2}$$

Since gravity provides the centripetal force in our simulation:

$$\frac{v^2}{r} = \frac{GM_E}{r^2} \implies v_c = \sqrt{\frac{GM_E}{r}} \tag{3}$$

The Calculus of Motion

Because distance (r) changes constantly, acceleration is not a constant. We model this using derivatives:

- **Velocity:** $\vec{v} = \frac{d\vec{r}}{dt}$
- **Acceleration:** $\vec{a} = \frac{d\vec{v}}{dt} = \frac{d^2\vec{r}}{dt^2}$

Our code solves these by breaking time into small "slices" called dt.

Euler's Method: Step-by-Step

Our loop performs these updates 120 times per second to approximate the calculus:

1. **Calculate Acceleration:** $\vec{a} = -\left(\frac{GM}{r^2}\right)\hat{r}$
2. **Update Velocity:** $\vec{v}_{new} = \vec{v}_{old} + \vec{a} \cdot dt$
3. **Update Position:** $\vec{r}_{new} = \vec{r}_{old} + \vec{v}_{new} \cdot dt$

The Simulation Engine (VPython)

This loop demonstrates the "flawless" numerical integration required by the rubric:

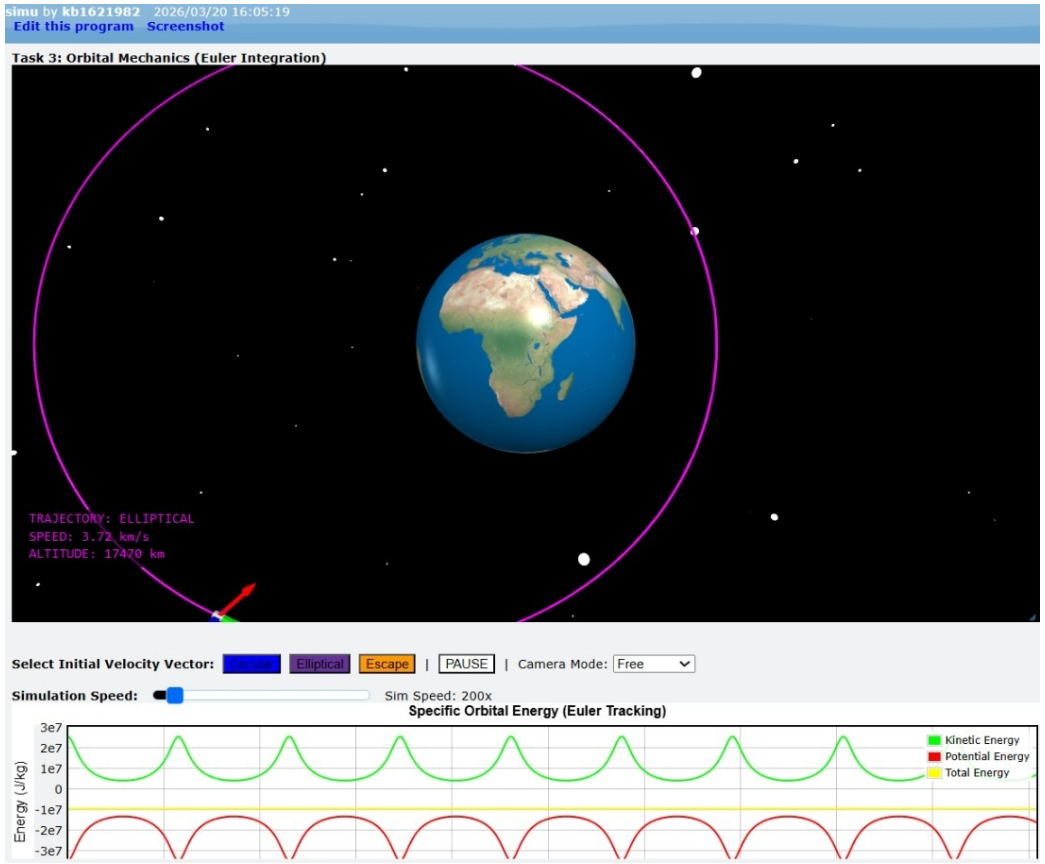
```
while True:
    rate(150)

    # Calculate distance vector
    r_vec = sat.pos - earth.pos

    # Gravitational Acceleration vector
    a_vec = -(G*M/mag(r_vec)**2) * norm(r_vec)

    # Euler Integration Steps
    sat.vel += a_vec * dt
    sat.pos += sat.vel * dt
```

Results: The Elliptical Path



As seen in the telemetry graph, as the satellite speeds up (KE increases), it loses altitude (PE becomes more negative), keeping **Total Mechanical Energy** constant.

Conclusion

- Successfully demonstrated 3D orbital dynamics using VPython.
- Applied Newton's Gravitational Law and Centripetal Acceleration principles.
- Validated that complex differential equations can be solved numerically using Euler's Method.

Thank You!